

Simulazioni n -body su GPU: approccio naive e Barnes-Hut

Relazione del progetto di GPGPU - Damiano Trovato

Abstract

Con la seguente relazione si documenta la realizzazione di simulazioni n -body tramite soluzioni parallele su GPU. Si trattano quindi aspetti degni di nota, quali gli algoritmi, i metodi numerici, le ottimizzazioni e un'analisi delle performance. Le soluzioni principali presentate saranno tre: un'approccio naive su GPU, un approccio approssimato Barnes-Hut totalmente su GPU (con quad-tree seriale) e un approccio approssimato Barnes-Hut ibrido GPU-CPU.

1 Introduzione

In questa sezione offriamo un'introduzione al progetto, dando una definizione del problema n -body, i dettagli fisici e matematici e come verranno sviluppati i test.

1.1 Simulazioni n -body

Una simulazione a n corpi permette di calcolare l'evoluzione temporale di un sistema costituito da, appunto, n corpi, che interagiscono tra loro con delle forze che dipendono dalla simulazione. Queste simulazioni sono fondamentali, in quanto ogni problema a n corpi non banale, e con configurazione non banale¹, non dispone di una soluzione analitica.

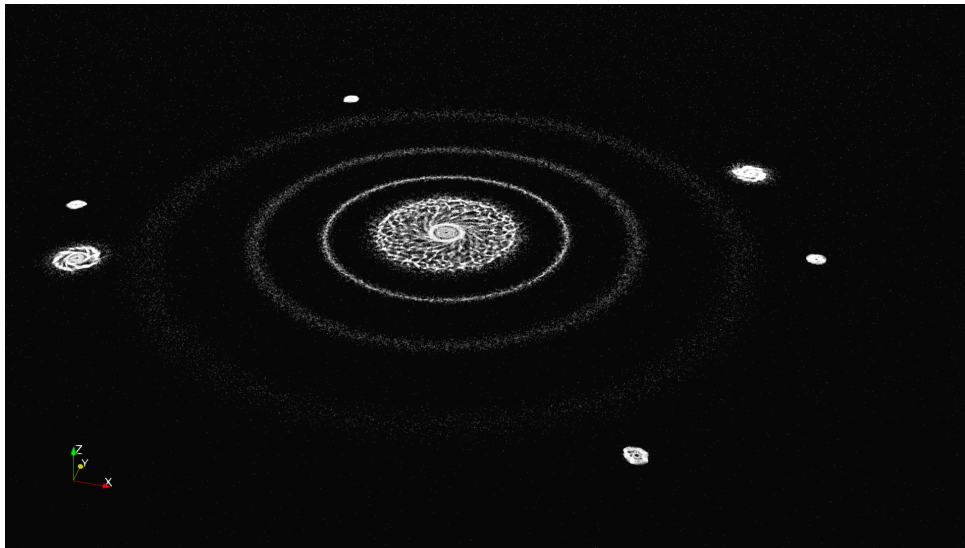


Immagine 1: Una demo a 180k corpi: <https://youtu.be/oSdXonzxpXk?si=4AL8uCfM6irI1BcW>

Sono fondamentali nell'ambito dell'astrofisica computazionale, offrendo soluzioni per osservare l'evoluzione di sistemi cosmologici.

¹assenza totale di forze, o configurazioni con determinate simmetrie.

1.2 Aspetti fisici e matematici

Si offre una lista delle formule matematiche che rendono fattibili le simulazioni realizzate all'interno del progetto. Le seguenti formule sono state prese dal video [?].

Legge di gravitazione universale La seguente formula detta l'attrazione che agisce tra due corpi rispettivamente di massa m_1 ed m_2 , situate a distanza $|r|$. Questa grandezza è direttamente proporzionale al prodotto delle masse e inversamente proporzionale al quadrato della distanza, ossia:

$$F = G \frac{m_1 m_2}{|r|^2} \hat{r} = G \frac{m_1 m_2}{|r|^2} \cdot \frac{r}{|r|} = G \frac{m_1 m_2}{|r|^3} r. \quad (1)$$

In natura G è una costante minuscola che rende l'attrazione tra i corpi rilevante solo se i corpi hanno massa elevatissima o distanza reciproca ridotta. Ai fini della simulazione, questa costante verrà portata a valori molto più grandi. $\hat{r}$ indica invece il versore² che punta da m_1 a m_2 , ottenuto normalizzando r . Sostituiamo quindi $\hat{r}$ con la sua formula esplicita.

Dalla forza all'accelerazione Supponiamo sul corpo m_1 agisca una determinata forza F :

$$F = m_1 a \Rightarrow a = \frac{F}{m_1} = G \frac{m_2}{|r|^3} r. \quad (2)$$

In definitiva, l'accelerazione che agisce su un corpo m_i è data da:

$$a_i = \sum_{j=0, j \neq i}^{n-1} G \frac{m_j}{|r_j|^3} r_j. \quad (3)$$

Dall'accelerazione alla velocità L'accelerazione è la derivata della velocità sul tempo, possiamo quindi calcolare la velocità di un corpo tramite

$$a = \frac{dv}{dt} \Rightarrow a dt = dv, \quad (4)$$

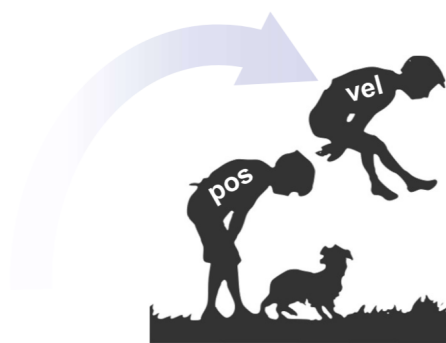
fissando un valore di $dt = \text{delta_time}$.

Dalla velocità allo spostamento

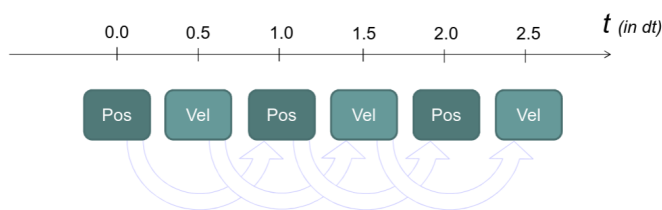
$$x = x_0 + v dt \quad (5)$$

dove $dt = \text{delta_time}$.

1.3 Metodo numerico di integrazione: leapfrog



3D video gaming - Marco Tarini
Univ Milano 2019/2020



Il metodo numerico utilizzato nella simulazione, è noto come metodo **leapfrog**, o metodo della cavallina. Il nome deriva dal modo in cui si alterna l'integrazione della velocità a quella del tempo.

Questo metodo numerico, sfasando di $dt/2$ l'integrazione della velocità e della posizione, ottiene errore ridotto, miglior comportamento asintotico e conservazione dell'energia del sistema, rispetto al metodo di Eulero. Sono tecniche di integrazione egualmente onerose, ma il metodo Leapfrog è più accurato e totalmente reversibile [?].

²vettore unitario

Pseudo-codice 1 Integrazione Leapfrog

```
1: Calcola l'accelerazione
2: Aggiorna la velocità per  $dt/2$ 
3: for Numero di iterazioni do
4:   Aggiorna la posizione per  $dt$ 
5:   Calcola l'accelerazione
6:   Aggiorna la velocità per  $dt$ 
7: end for
```

1.4 Studio delle performance su hardware

I test delle varie soluzioni fornite, saranno effettuate su tre schede video differenti, ossia

- **AMD Radeon RX 6600 CLD 8GB** (discreta);
- **Intel Core Ultra 9 285H - Intel Arc Pro 130T/140T** (integrata);
- **AMD Ryzen 7 7700 - gfx1036** (integrata), ma questa verrà usata solo sull'algoritmo naive, perché le prestazioni sull'algoritmo Barnes-Hut sono disastrose e poco interessanti da studiare.

In tutte le soluzioni affrontate, verranno omesse le operazioni di scrittura a disco, che rappresentano una grande fonte di overhead, ma che sono anche fuori dallo scope del progetto.

Le nostre valutazioni avverranno rispetto ad un numero crescente di corpi n da 10 a ~ 180000 . Per ogni simulazione, valutiamo il calcolo di 10 frame, in modo da ridurre l'effetto della varianza sul singolo tempo di esecuzione del kernel, su un frame particolarmente fortunato (o sfortunato).

2 Simulazione parallela naive su GPU

Il primo approccio alla simulazione di sistemi a n corpi che si è deciso di realizzare, è una soluzione parallela basata sull'algoritmo naive, con complessità computazionale $\mathcal{O}(n^2)$. Consiste banalmente nel calcolo brute-force di tutte le interazioni corpo-a-corpo.

2.1 Kernel fondamentali e ottimizzazioni

Calcolo dell'accelerazione naive Il primo kernel che trattiamo, viene eseguito su ogni corpo per ogni istante della simulazione, e permette il calcolo dell'accelerazione su ogni corpo. dalla teoria all'implementazione,

Pseudo-codice 2 Kernel per il calcolo dell'accelerazione di un corpo

Require: array dei corpi C , $n = |C|$

```
1:  $i \leftarrow$  id globale del work-item
2: if  $i \geq n$  then return  $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input
3: end if
4:  $a \leftarrow 0$ 
5: for all  $j \in \{0, \dots, n-1\}$  do
6:   if  $i = j$  then
7:     continue
8:   end if
9:    $r \leftarrow C_{pos}[i] - C_{pos}[j]$ 
10:   $r = \max(r, \varepsilon)$ 
11:   $a \leftarrow G \frac{C_{mass}[j]}{|r|^3} r + a$ 
12: end for
13:  $C_{acc}[i] \leftarrow a$ 
```

subentrano delle ottimizzazioni. Il calcolo di $\frac{1}{|r|^3}$ è infatti ottimizzato col calcolo della radice quadrata inversa su r al cubo (notoriamente più veloce del calcolo esplicito della radice quadrata e della divisione).

```
const float2 dist_vec = body_pos[i] - this_pos;
const float dist_squared = fmax(dot(dist_vec, dist_vec), EPSILON);
const float inv_dist = rsqrt(dist_squared);
const float inv_dist3 = inv_dist * inv_dist * inv_dist;
this_acc += G * body_mass[i] * dist_vec * inv_dist3;
```

Osserviamo inoltre una funzione $\max(r, \varepsilon)$, con ε valore arbitrariamente piccolo per evitare che $|r|^3 \rightarrow 0$ (con conseguente accelerazione $\rightarrow \pm\infty$) o peggio ancora $|r|^3 = 0$, che causerebbe un errore di divisione per zero. Questo problema si pone perché nella simulazione non consideriamo collisioni, i corpi possono passarsi attraverso, ottenendo valori molto piccoli di r . Scegliamo il valore ε sperimentalmente.

Calcolo della velocità Il seguente kernel è invece dedicato al calcolo della velocità dall'accelerazione. Il delta tempo dt è un argomento, in quanto necessario per il metodo Leapfrog.

Pseudo-codice 3 Kernel per il calcolo della velocità

Require: array dei corpi C , $n = |C|$, delta tempo dt

```
1:  $i \leftarrow$  id globale del work-item
2: if  $i \geq n$  then return  $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input
3: end if
4:  $C_{vel}[i] \leftarrow C_{vel}[i] + C_{acc}[i] \cdot dt$ 
```

Calcolo della posizione L'ultimo kernel della simulazione naive è dedicato all'aggiornamento della posizione.

Pseudo-codice 4 Kernel per il calcolo della posizione

Require: array dei corpi C , $n = |C|$, delta tempo dt

- 1: $i \leftarrow$ id globale del work-item
 - 2: **if** $i > n$ **then return** $\triangleright$ Usciamo dal programma se il work-item è fuori dal dominio dell'input
 - 3: **end if**
 - 4: $C_{pos}[i] \leftarrow C_{pos}[i] + C_{vel}[i] \cdot dt$
-

2.2 Codice host, algoritmo complessivo

Pseudo-codice 5 Simulazione n -body naive

Require: array dei corpi C , $n = |C|$, delta tempo dt , numero di iterazioni t

- 1: lancia n kernel **accelerazione naive** su C con delta tempo dt
 - 2: lancia n kernel **calcolo velocità** su C con delta tempo $dt/2$
 - 3: **for** t **do**
 - 4: lancia n kernel **calcolo posizione** su C con delta tempo dt
 - 5: lancia n kernel **accelerazione naive** su C con delta tempo dt
 - 6: lancia n kernel **calcolo velocità** su C con delta tempo dt
 - 7: **end for**
-

In realtà, per ogni kernel lanciato, il numero di kernel è quasi sempre arrotondato al primo multiplo di 32 più grande di n , per evitare che le performance vengano penalizzate (aggiungerei, in maniera disastrosa) da un numero primo di particelle.

2.3 Numero di accessi in memoria globale

Per avere una prospettiva più consapevole nello studio delle prestazioni dell'algoritmo, andiamo a osservare il numero di accessi in memoria globale, che sappiamo rappresentare una delle più grandi fonti di bottleneck nell'esecuzione dei kernel degli programmi su GPU. Considerando un solo kernel, questo effettua il seguente numero di accessi in memoria globale:

- **aggiornamento posizione**, uno alla velocità e uno alla posizione, $\mathcal{O}(1)$;
- **aggiornamento velocità**, uno all'accelerazione e uno alla velocità, $\mathcal{O}(1)$;
- **aggiornamento accelerazione**, n alle posizioni, n alle masse, 1 all'accelerazione, $\mathcal{O}(n)$;

con n numero di corpi.

2.4 Benchmarking

Valutiamo finalmente le prestazioni dell'algoritmo. Dai grafici delle successive pagine, emergono un paio di osservazioni: entrambe le GPU integrate performano in maniera simile sul calcolo di posizione e velocità. In generale, il calcolo di posizione e velocità sembra crescere linearmente in funzione del numero di corpi.

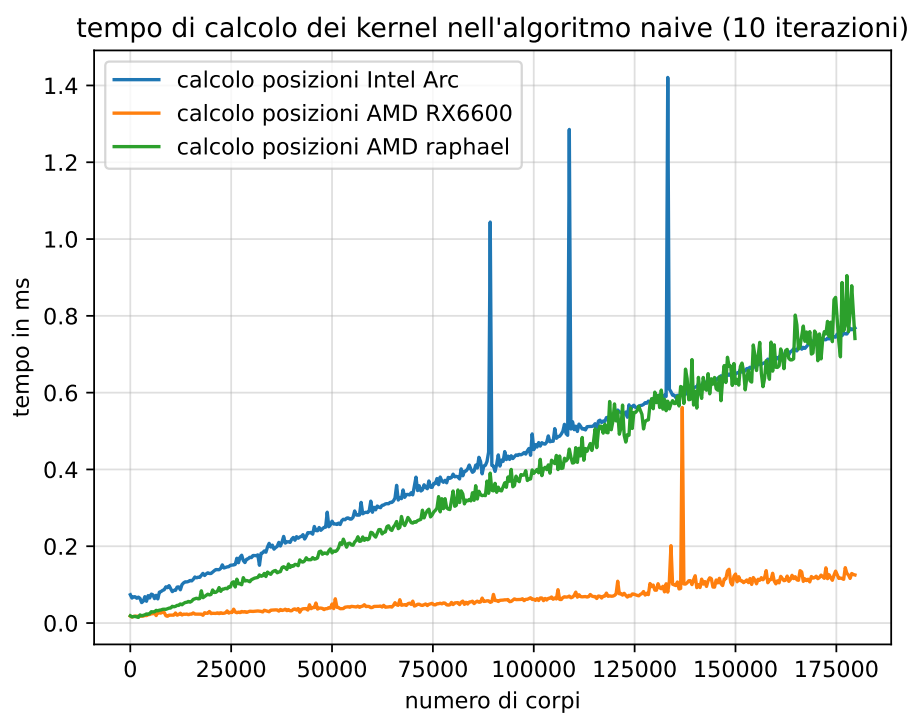


Immagine 2: Tempo di calcolo della posizione, in funzione del numero di corpi, valutato su tre schede video differenti. In arancione, abbiamo l'unica GPU discreta, la RX6600 di casa AMD.

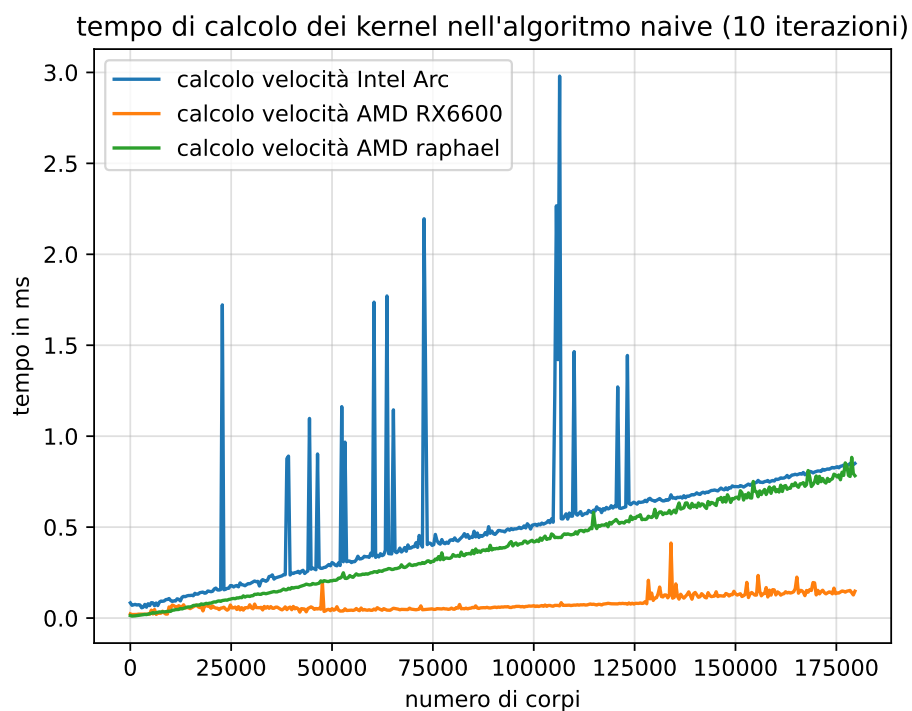


Immagine 3: Tempo di calcolo della velocità, in funzione del numero di corpi.

Meno fortunato è il calcolo dell'accelerazione dei corpi. L'alto numero di operazioni e di accessi alla memoria globale, demolisce totalmente le performance della grafica integrata del Ryzen 7 7700 (Raphael), meno avanzata se messa a confronto con le performance della grafica integrata Intel.

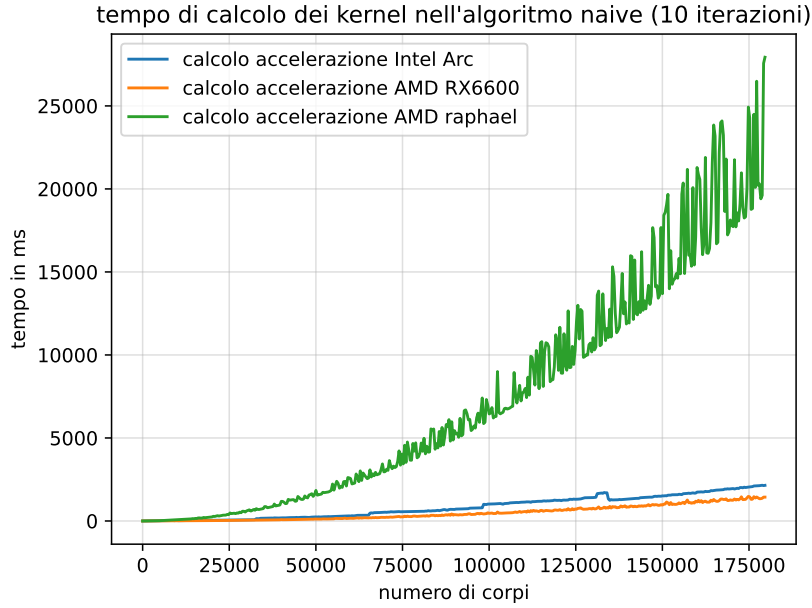


Immagine 4: Tempo di calcolo dell'accelerazione, in funzione del numero di corpi.

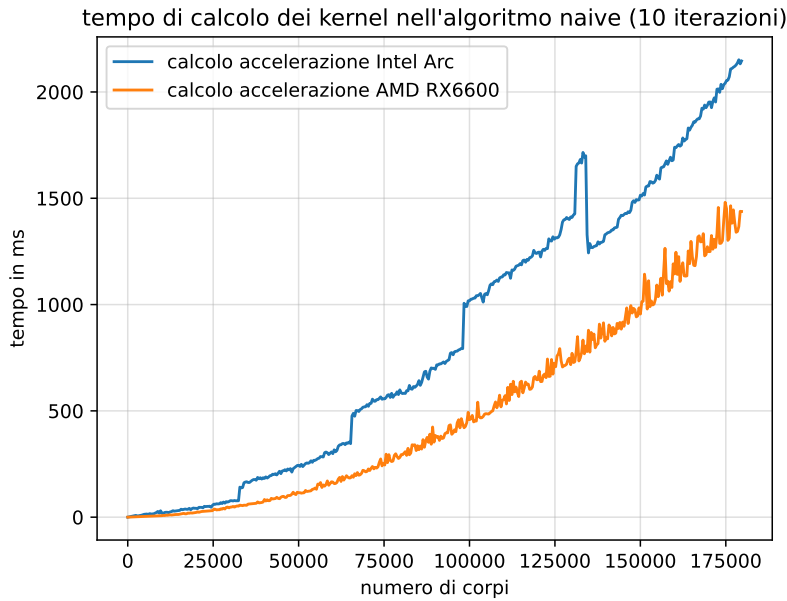


Immagine 5: Ai fini di visualizzazione, proponiamo lo stesso grafico precedente escludendo l'AMD Raphael.

Seppur tutte le visualizzazioni dei sistemi siano state effettuate in un secondo momento all'interno del software ParaView, è possibile calcolare il tempo massimo ammesso per il calcolo di un fotogramma della simulazione, in contesto realtime. La formula è

$$t = \frac{1000}{\text{frame per secondo}} \text{ ms}, \quad (6)$$

per cui, i tempi per calcolare un frame a 30 FPS dovranno essere al più $t_{30} = 33,33 \text{ ms}$, per 60 FPS al più la metà, ossia $t_{60} = 16,6 \text{ ms}$. Moltiplichiamo queste quantità per 10, per poter riutilizzare i dati già elaborati.

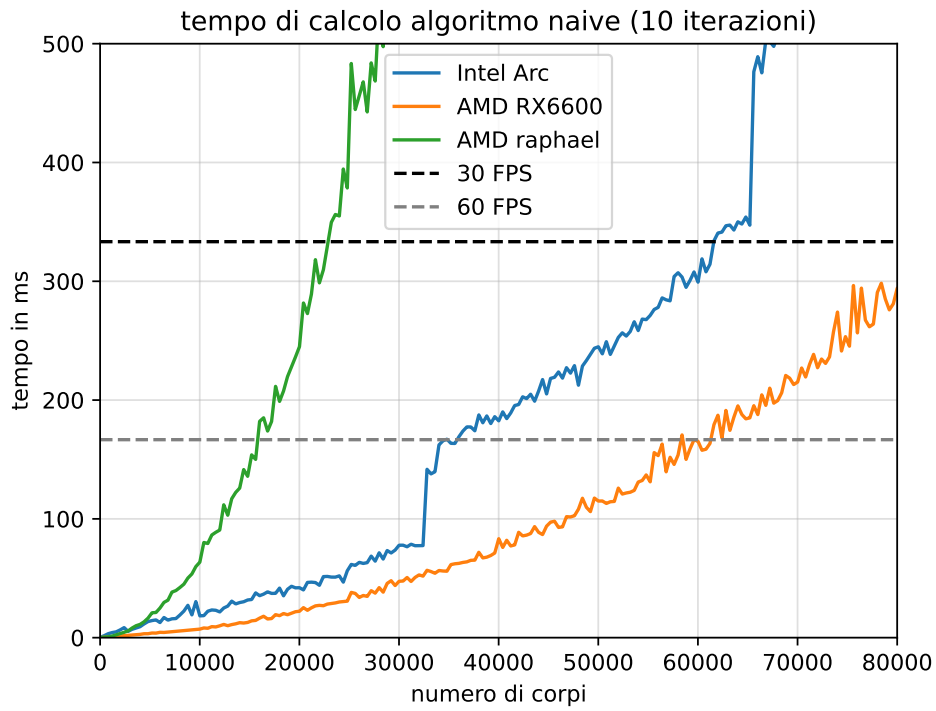
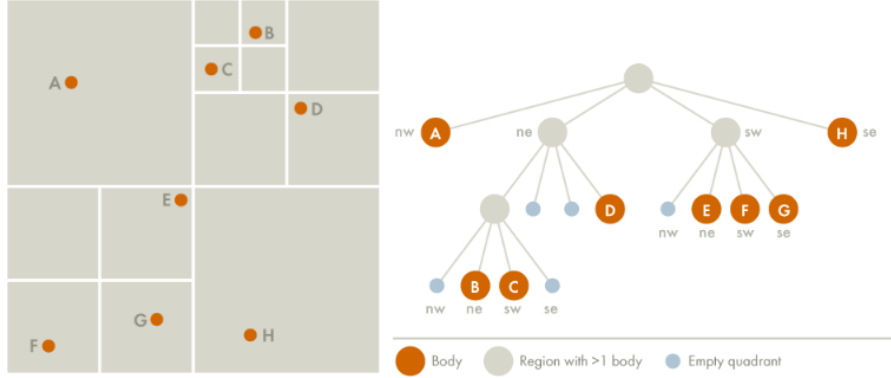


Immagine 6: Somma dei tempi dei kernel

A 30 FPS, Raphael AMD può simulare 22800 corpi, l'Intel ARC fino a 61200 corpi e l'AMD RX6600 fino a 82800 corpi. A 60 FPS, Raphael non supera i 15610 corpi, l'Intel ARC i 34400 corpi e l'AMD RX6600 i 58000 corpi. La crescita quadratica dell'algoritmo naive (sia in termini di algoritmo, che in termini di accessi alla memoria globale) è il punto debole dell'algoritmo.

3 Simulazione Barnes-Hut totalmente su GPU con costruzione del seriale quadtree

L'algoritmo Barnes-Hut permette di simulare sistemi a n corpi in maniera approssimata, con precisione arbitraria e complessità asintotica $\mathcal{O}(n \log n)$. Sfrutta una particolare struttura dati, ossia un quad-tree (figura 3), un albero in cui ogni nodo ha quattro figli, per suddividere ricorsivamente lo spazio, memorizzare valori relativi al sistema e velocizzare il passaggio più oneroso della simulazione brute-force: il calcolo delle forze (o dell'accelerazione), notoriamente un passaggio a complessità $\mathcal{O}(n^2)$. L'idea è la



seguente: costruiamo un albero che descrive l'intero sistema, e invece di calcolare l'attrazione tra un corpo c e il resto dei corpi C interrogandoli uno ad uno, si visita l'albero, e per ogni altro nodo ci chiediamo “posso approssimare tutti i figli di questo nodo ad un unico corpo?”. Lo **sviluppo in multipoli** ci dice che questa approssimazione è affidabile quando l'area che include questi corpi non è molto grande e quando la distanza da c è elevata: useremo un parametro θ soglia, che verrà confrontato proprio al rapporto tra le grandezze appena citate. Quando il criterio è rispettato, si accetta il nodo come approssimazione di tutti i corpi della regione associata, con posizione uguale al centro di massa, e massa uguale alla somma delle masse, altrimenti si continua la visita nell'albero. Su sistemi con molteplici regioni dense, ma molto distanti tra loro, l'algoritmo Barnes-Hut performa estremamente bene.

Tuttavia l'algoritmo paga l'overhead della costruzione del quad-tree, che richiede non pochi passaggi:

1. Definizione dell'area dell'intero sistema particellare, ossia la **bounding-box**.
2. Inserimento di ciascun corpo all'interno dell'albero.
3. Propagazione nei nodi interni delle masse e dei centri di massa dei rispettivi figli.

Questi passaggi sono i più costosi dell'algoritmo, ma sono anche alla base del suo vantaggio asintotico.

3.1 I kernel fondamentali

Calcolo dei margini della bounding-box I primi kernel sono quelli dedicati alla definizione della dimensione della bounding-box (ossia della radice del quad-tree). Usiamo una riduzione parallela per trovare il minimo e il massimo dell'intero sistema particellare, sia su x che su y . In questo modo, sarà possibile costruire un quadrato (la bounding-box) che conterrà l'intero sistema.

Questo kernel sarà lanciato fino a quando il minimo (massimo) si troverà alla posizione 0 dell'ultimo array usato. Precisamente, verrà lanciato per $\lceil \log_2(n) \rceil$ volte, con n numero di particelle.

Nel codice host, abbiamo deciso di usare due array, scambiando ogni volta l'array di input e quello d'output (in una sorta di ping-pong tra gli array per ridurre al minimo la memoria usata).

Costruzione del quad-tree Il secondo kernel che presentiamo, è quello relativo alla **costruzione del quad-tree**. Prima di presentare il kernel, definiamo in cosa consiste l'implementazione del nostro quad-tree. Non possiamo usare un approccio OOP o degli array di struct, ma possiamo creare degli array, uno per ciascuno dei campi dei nodi dell'albero. Otteniamo quindi

```
int cell_children = figli di ciascuna nodo. 4 volte più lungo degli altri;  
float2 cell_center = posizione del centro della cella del nodo;  
float cell_half_size = metà lunghezza della cella associata al nodo;  
float2 cell_center_of_mass = centro di massa della nodo;  
float cell_mass = massa del nodo;
```

Pseudo-codice 6 Kernel per la riduzione parallela (ricerca del minimo o massimo)

Require: array A , array B , dimensione dell'input n , array $BoundingBox$, flag per l'ultima riduzione

```
1:  $j \leftarrow$  id globale del work-item
2:  $i \leftarrow 2j$ 
3: if  $i > n$  then return  $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input
4: end if
5:  $val \leftarrow A[i]$ 
6: if  $i + 1 < n$  then
7:    $val \leftarrow \min(val, A[i + 1])$   $\triangleright \max(val, A[i + 1])$  nella ricerca del massimo
8: end if
9: if è l'ultima riduzione then
10:    $BoundingBox[MIN\_INDEX] \leftarrow val$   $\triangleright BoundingBox[MAX\_INDEX]$  nella ricerca del massimo
11: end if
12:  $B[j] \leftarrow val$ 
```

La struttura gerarchica è permessa dal primo array, `cell_children`, che contiene in posizione $4i + j$ il j -esimo figlio del nodo i . Stabiliamo inoltre la seguente convenzione per associare un indice a ogni quadrante della rispettiva cella:

```
/* CONVENZIONE:
   10 11
   00 01
*/
inline int get_quadrant(float2 pos, float2 center) {
    int right = pos.x >= center.x;
    int top = pos.y >= center.y;
    return (top << 1) | right;
}
```

Inoltre, l'array `cell_children` contiene al suo interno riferimenti a entità di tipi differenti, ossia nodi e corpi. Stabiliamo la convenzione per cui tutti gli indici che fanno riferimento a nodi dell'albero, hanno indice maggiore o uguale al numero di corpi. Questo significa che, quando memorizziamo l'indice ad un nodo, alla sua posizione del quad-tree va sommato il numero di corpi. Quando invece vogliamo usare l'indice per accedere ad un altro nodo, dobbiamo prima sottrarre la stessa quantità.

In letteratura non si trovano esempi di algoritmi di costruzione di quad-tree paralleli su GPU senza ausilio delle funzioni atomiche: usiamo quindi un approccio seriale, mantenendo tuttavia i dati su GPU, per risparmiare sullo scambio dati CPU-GPU. Ci rifaremo alla costruzione del quad-tree dell'algoritmo originale.

Reset e inizializzazione dell'albero Questo kernel si occupa del reset della struttura ad albero. Vengono posti nulli i valori del centro di ciascun nodo e dell'`halfsize`, a -1 le masse e gli indici dei figli. Il kernel con $id = 0$ fa uno step in più, ossia settare la posizione del centro e l'`halfsize` in funzione del risultato della riduzione. Questo kernel in realtà viene eseguito prima della costruzione del quadtree, ma ai fini della spiegazione, abbiamo preferito introdurlo dopo. Viene lanciato per ciascun nodo dell'albero.

Propagazione delle masse e dei centri di massa In letteratura gli approcci ottimali per la summarize-tree sfruttano spesso funzioni atomiche; tuttavia, è possibile implementare un kernel da lanciare più volte per compiere lo stesso lavoro.

Il kernel funziona in questo modo: dato un nodo A , calcolane la massa (e il centro di massa) osservandone i figli. Se uno di questi figli ha valori non inizializzati (questo avviene quando non si è ancora calcolata la loro massa), esci dal kernel e non modificare la massa del nodo (e nemmeno il centro di massa). Questo kernel va lanciato sull'intero quad-tree per un numero di volte pari alla sua altezza, ottenendo una sorta di "onda" di pesi che si propagano dal basso verso l'alto.

Pseudo-codice 7 Summarize tree (propagazione masse e centri di massa)

Require: array del quad-tree QT , array dei corpi C

```
1:  $i \leftarrow$  id globale del work-item
2: if  $i \geq \text{len}(QT)$  then return
3: end if
4:  $total\text{-}mass \leftarrow 0$ 
5:  $center\text{-}of\text{-}mass \leftarrow (0, 0)$ 
6:  $weighted\text{-}pos \leftarrow (0, 0)$ 
7: for all figli di  $QTree[i]$  do
8:    $j \leftarrow$  indice del figlio
9:   if  $j$  è un indice valido then
10:      $total\text{-}mass \leftarrow total\text{-}mass + QT[j]_{mass}$ 
11:      $weighted\text{-}pos \leftarrow QT[j]_{pos} \cdot QT[j]_{mass}$ 
12:   else
13:     return
14:   end if
15: end for
16:  $QT[i]_{mass} \leftarrow total\text{-}mass$ 
17:  $QT[i]_{center\text{-}of\text{-}mass} \leftarrow weighted\text{-}pos/total\text{-}mass$ 
```

In realtà, nelle istruzioni 9-15 va considerato pure un controllo sul tipo di figlio. j potrebbe infatti essere un indice ad un nodo o un indice ad un corpo. Nel primo caso il calcolo coinvolgerà l'accesso ad un altro slot del quad-tree, nel secondo caso ad uno slot dell'array dei corpi C .

Calcolo dell'accelerazione usando il quad-tree Questa parte dell'algoritmo è imbarazzantemente parallelizzabile: tuttavia, l'algoritmo di attraversamento di un albero come il quad-tree, se non implementabile tramite ricorsione, richiede l'utilizzo di uno stack, il che rende questo kernel un altro elemento interessante.

Il kernel va lanciato su ciascuno dei corpi (chiameremo il nostro corpo $C[i]$), e procede nel seguente modo:

1. si inserisce nello stack la radice del quad-tree (ossia l'intero sistema);
2. finché lo stack non è vuoto, si estrae un elemento dallo stack;
3. se l'elemento è un corpo, calcola regolarmente l'accelerazione indotta da quest'ultimo su $C[i]$. Se il corpo estratto è $C[i]$ stesso, si ignora;
4. se l'elemento è un nodo associato a una cella che **non** contiene $C[i]$, possiamo verificare la condizione

$$\frac{\text{dimensione della cella}}{\text{distanza dal centro di massa della cella}} < \theta, \quad (7)$$

se questa è vera, allora possiamo calcolare l'accelerazione sull'intero albero che ha come radice il nodo in questione usando la sua massa e il suo centro di massa come posizione;

5. se nessuna tra le condizioni in 3 e 4 sono rispettate, allora si inseriscono i quattro figli del nodo nello stack;

La condizione relativa al coefficiente θ può essere riformulata in modi equivalenti, che permettono di risparmiare su operazioni di radici quadrate e divisioni. Nel nostro caso, useremo

$$(\text{dimensione della cella})^2 < (\text{distanza dal centro di massa della cella})^2 \cdot \theta^2. \quad (8)$$

Aggiornamento di velocità e posizione Questi ultimi due kernel sono uguali a quelli del metodo naive, quindi non verranno approfonditi ulteriormente.

3.2 L'algoritmo complessivo

3.3 Numero di accessi in memoria globale

Riproponiamo di nuovo le osservazioni sul numero di accessi in memoria globale:

- **riduzione parallela (minimo e massimo)**, 2 accessi alla posizione, ma il kernel viene rilanciato $\log n$ volte, quindi $\mathcal{O}(\log n)$;

- **costruzione dell'albero**,
- **propagazione dei pesi e dei centri di massa**, ogni kernel viene rilanciato per `MAX_TREE_DEPTH` volte, e si accede sempre a 2 campi dei 4 figli. $\mathcal{O}(1)$;
- **aggiornamento accelerazione**, dipende pesantemente dalla distribuzione di partenza. COn
- **aggiornamento velocità**, uno all'accelerazione e uno alla velocità, $\mathcal{O}(1)$;
- **aggiornamento posizione**, uno alla velocità e uno alla posizione, $\mathcal{O}(1)$;

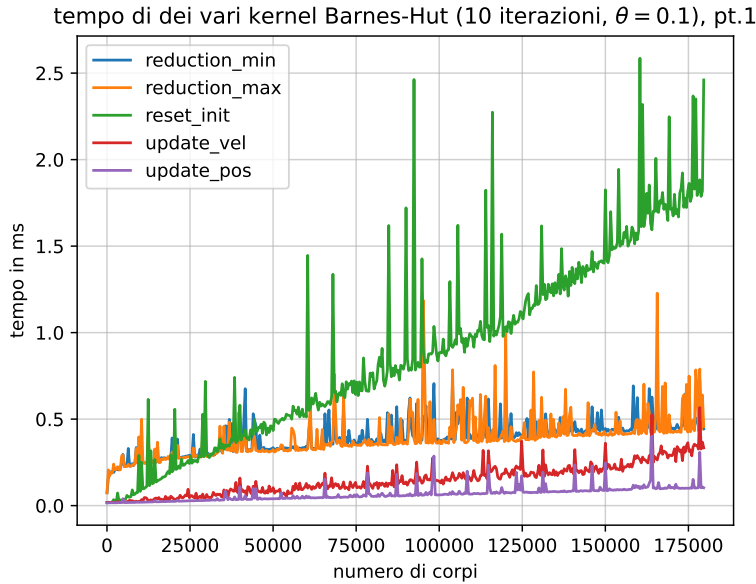
con n numero di corpi.

3.4 Benchmarking

La nostra implementazione dell'algoritmo Barnes-Hut si compone di molteplici kernel:

1. riduzione per trovare il minimo;
2. riduzione per trovare il massimo;
3. reset dell'albero;
4. costruzione dell'albero;
5. propagazione delle masse e dei centri di massa nell'albero;
6. calcolo dell'accelerazione;
7. aggiornamento della velocità;
8. aggiornamento della posizione.

Tra questi passaggi, 1, 2, 3, 7 e 8 risultano essere quelli meno onerosi della nostra implementazione. Fissiamo $\theta = 0.1$ e scegliamo come GPU di riferimento la AMD RX6600. In figura 3.4, il kernel più oneroso appare essere quello di reset dell'albero, il che risulta riconducibile all'alto numero di accessi in memoria richiesti.



I passaggi 4, 5 e 6 sono invece molto più onerosi dei precedenti. In particolare, nella nostra implementazione, la costruzione dell'albero (4) seriale è un vero e proprio collo di bottiglia per il resto dell'algoritmo. La propagazione delle masse (5) non cresce in maniera significativa ($\sim 273ms$ su ~ 180000 particelle), ma più velocemente dei kernel precedenti. Citiamo per ultimo il kernel dedicato al calcolo dell'accelerazione (6): l'intero algoritmo Barnes-Hut punta sull'ottimizzazione di questo passaggio tramite l'approssimazione sensibile al parametro θ . Ne consegue che l'algoritmo vada valutato anche in funzione di quest'ultimo. Se in figura 7 abbiamo i kernel 4,5 e 6 con $\theta = 0.1$, gli stessi sono messi a confronto con $\theta = 0.5$ in figura 8. Osserviamo un importante guadagno in prestazioni, che sottolinea la potenza dell'approssimazione Barnes-Hut.

Va tuttavia specificata una cosa quando desideriamo mettere a confronto l'approccio naive a quello Barnes-Hut: il vantaggio asintotico diventa un vantaggio effettivo solo con un numero di particelle molto alto. A causa delle risorse ridotte a nostra disposizione (e al bottleneck del build-tree seriale), non ci sarà possibile allocare abbastanza particelle per assistere al vantaggio di BH. Tuttavia, decidendo di ignorare il tempo degli altri kernel, possiamo mettere a confronto esclusivamente i tempi di calcolo delle accelerazioni nei rispettivi algoritmi. Dalla figura 9 emerge come la scelta del valore di θ sia cruciale per ottenere lo speedup promesso dall'algoritmo, molto più della scelta dell'hardware: per $\theta = 0.1$ il calcolo dell'accelerazione è più lento che nella simulazione naive. Per $\theta = 0.5$, cresce in maniera simile alla summarize tree.s

TODO inserire il confronto per $\theta = 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8$

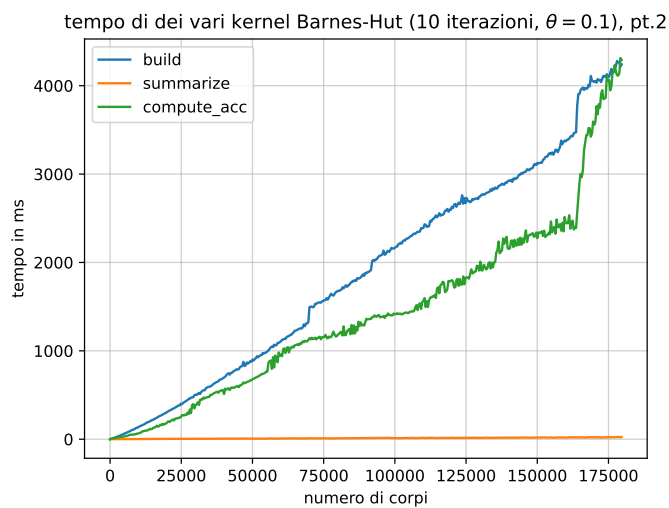


Immagine 7: text

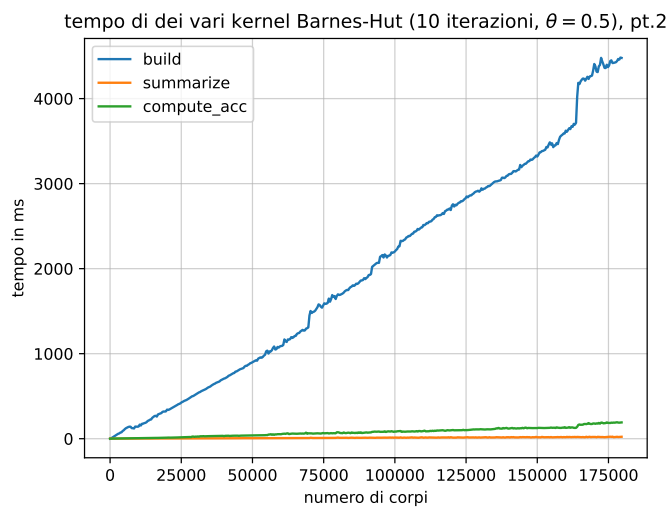


Immagine 8: text

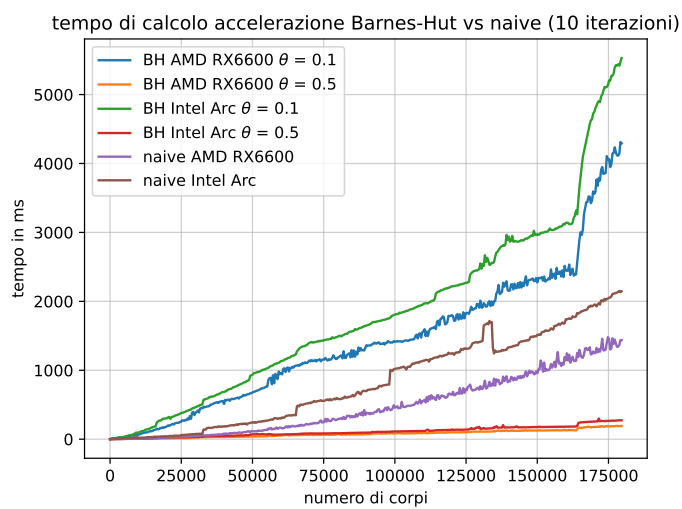


Immagine 9: Tempo...

4 Simulazione Barnes-Hut ibrida

La costruzione seriale del quad-tree rappresenta la principale fonte di overhead nella nostra attuale implementazione dell'algoritmo Barnes-Hut su GPU. Per ridurre al minimo i costi di trasferimento della memoria da CPU a GPU e viceversa, abbiamo affrontato il problema mantenendo tutti i dati su GPU, e usando un kernel eseguito da un solo work-item per la costruzione del quad-tree. Sfruttiamo finalmente il meglio della GPU e della CPU, affidando proprio alla seconda la costruzione dell'albero.

4.1 Benchmarking

Con questa implementazione ibrida dell'algoritmo Barnes-Hut, riusciamo finalmente a ottenere un vantaggio sull'approccio naive visibile col numero attuale di particelle coinvolte. In figura 10 mettiamo a confronto

- Algoritmo naive su Intel Arc;
- Algoritmo naive su AMD RX6600;
- Algoritmo naive su AMD Raphael;
- Algoritmo Barnes-Hut puro su GPU, Intel Arc, $\theta = 0.5$;
- Algoritmo Barnes-Hut puro su GPU, AMD RX6600, $\theta = 0.5$;
- Algoritmo Barnes-Hut ibrido CPU-GPU, Intel Arc, $\theta = 0.5$;
- Algoritmo Barnes-Hut ibrido CPU-GPU, AMD RX6600, $\theta = 0.5$.

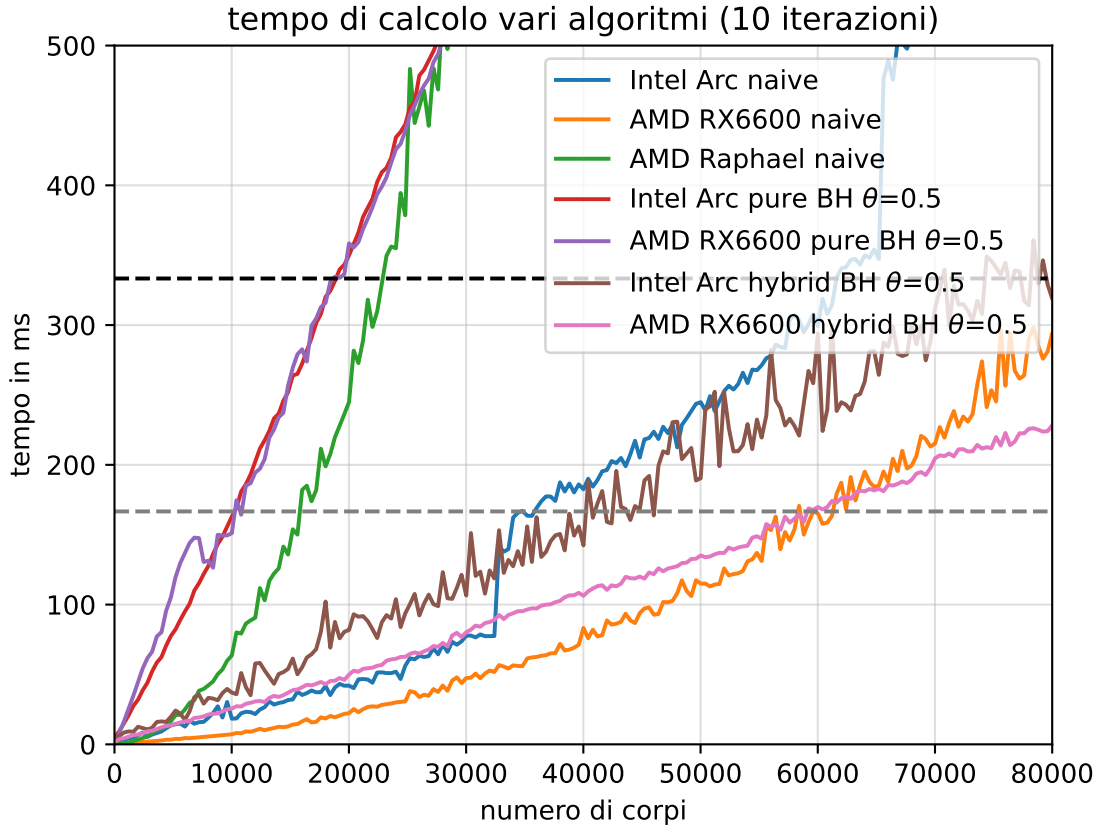


Immagine 10:

Questo approccio ibrido rende la nostra implementazione su Barnes-Hut viabile per applicazioni real-time. In particolare, questo vantaggio si ottiene a numeri molto più bassi di particelle sulla Intel Arc rispetto all'AMD RX6600, il che è riconducibile a due motivi fondamentali: un processore più performante, e l'uso della mappatura in memoria (molto conveniente sulle GPU integrate).